MEDUMBA.AI

MVP ARCHITECTURE DOCUMENT

AI-Enabled Preservation and Documentation of the Medumba Language

Version 1.0 — March 2026

CONFIDENTIAL

TABLE OF CONTENTS

1. Executive Overview
2. System Architecture Overview
3. Component 1: Learning App (Duolingo-Style Web & Mobile)
4. Component 2: Dictionary Module & Community Word Harvesting
5. Component 3: AI Module, Chatbot & TeachAnything.ai Integration
6. Data Architecture & Storage
7. API Design
8. Infrastructure & Deployment
9. Security Architecture
10. Integration Map: How the Three Components Connect
11. Development Roadmap

1. EXECUTIVE OVERVIEW

Medumba.ai is an AI-enabled language preservation and learning platform for Medumba, an endangered Grassfields Bantu language spoken in the Ndé Division of Cameroon's West Region. The MVP is architected as **three interconnected components** that together deliver a complete language learning, preservation, and AI-assisted teaching ecosystem:

Component	What It Does	Key Capabilities
Learning App	Duolingo-style language learning experience for web and mobile	Structured lessons, gamified vocabulary drills, pronunciation practice with AI scoring, spaced repetition, adaptive difficulty, cultural storytelling modules, progress tracking, leaderboards
Dictionary Module	Living trilingual dictionary and community word harvesting engine	4,000+ Medumba–French–English entries, 600+ elder pronunciation files, community word submission workflow, cultural context annotations, search/browse/filter, XML/JSON corpus export
AI Module	Intelligence layer powering NLP, chatbot, and TeachAnything.ai integration	Medumba NLP engine, tonal pronunciation evaluation, adaptive learning algorithms, bilingual chatbot, TeachAnything.ai open-access integration, content generation pipeline

Design Principles: The architecture follows four governing principles: (1) **Community-first** — the most durable deliverables (dictionary, elder recordings, teacher network) are completed first and stand alone regardless of the technology layer; (2) **Open-core model** — publicly accessible components under Apache 2.0/MIT licenses, proprietary AI engine and subscription systems retained by the Foundation; (3) **Offline-capable** — mobile app functions in low-connectivity environments across the Ndé Division; (4) **Cloud-native** — automatic scaling, reduced maintenance burden, and AI-driven monitoring.

2. SYSTEM ARCHITECTURE OVERVIEW

The Medumba.ai MVP uses a layered architecture with clear separation between presentation, application logic, AI services, and data storage. All three user-facing components share a common backend API layer and data infrastructure.

2.1 Architecture Layers

Layer	Technology	Purpose
Presentation — Web	React.js 18+, Tailwind CSS	Responsive web app for Learning App and Dictionary; progressive web app (PWA) capabilities for light offline use
Presentation — Mobile	Flutter (Dart)	Cross-platform iOS/Android native app with offline-first architecture;
API Gateway	Node.js / Express.js	RESTful API layer serving all three components; JWT authentication; rate limiting; request routing to microservices
Application Services	Node.js + Python (FastAPI)	Business logic for lessons, dictionary CRUD, user management, subscription billing; Python services handle AI/ML workloads
AI / ML Services	Python (FastAPI), PyTorch, Whisper, LangChain	NLP engine, pronunciation scoring, adaptive learning algorithms, chatbot orchestration, TeachAnything.ai connector
Data Layer	PostgreSQL, MongoDB, Redis, S3-compatible object storage	Relational data (users, progress, billing), document store (dictionary entries, linguistic corpus), cache, audio/media files
Infrastructure	Docker, Kubernetes (managed), CDN, CI/CD	Containerized deployment, auto-scaling, global content delivery, automated testing and deployment pipeline

2.2 High-Level Data Flow

The three components interact through a unified backend:

- The Learning App requests lesson content, submits pronunciation audio for scoring, and writes progress data through the API Gateway
- The Dictionary Module reads/writes dictionary entries, serves pronunciation files from object storage, and processes community word submissions through a moderation queue
- The AI Module provides services consumed by both the Learning App (pronunciation scoring, adaptive difficulty, content generation) and the chatbot (conversational NLP, TeachAnything.ai bridge)
- TeachAnything.ai connects via a dedicated integration API that exposes Medumba lesson content and chatbot capabilities while maintaining data sovereignty under the IP Framework

3. COMPONENT 1: LEARNING APP (DUOLINGO-STYLE WEB & MOBILE)

The Learning App is the primary interface through which diaspora learners interact with Medumba.ai. It delivers structured, gamified language learning modeled on the Duolingo paradigm but purpose-built for the unique challenges of a **five-tone, low-resource African language** where standard speech recognition models do not exist and cultural context is inseparable from language acquisition.

3.1 User Experience Architecture

- Onboarding flow: language proficiency self-assessment (beginner/heritage/intermediate), learning goal selection (conversational/cultural/pronunciation), daily commitment preference (5/10/15/20 min)
- Home screen: daily lesson queue, streak counter, XP progress, leaderboard position, cultural story of the day
- Lesson flow: 5–8 minute micro-lessons mixing vocabulary introduction, listening comprehension, pronunciation practice, and contextual usage
- Review system: spaced repetition algorithm (modified SM-2) surfaces words at optimal intervals based on individual performance data
- Gamification layer: XP points, daily streaks, achievement badges, weekly leaderboards, level progression (Seedling → Sprout → Tree → Baobab → Elder)

3.2 Lesson Module Architecture

Lessons are structured as JSON lesson packs stored in the content database and served dynamically based on the learner's adaptive path:

- Vocabulary Unit: word introduction with Medumba audio (elder recording), French/English translations, example sentence, cultural context note, tonal notation
- Listening Challenge: play elder pronunciation → learner selects correct translation from 4 options; difficulty adapts based on accuracy
- Pronunciation Challenge: learner records spoken Medumba → AI pronunciation scoring engine returns tonal accuracy score + specific feedback on tone errors
- Sentence Builder: drag-and-drop Medumba words into correct sentence order; teaches noun class agreement and basic grammar
- Cultural Story Module: illustrated mini-narrative using lesson vocabulary; elder voice narration with synchronized text highlighting
- Proverb Exploration: Medumba proverb with literal translation, cultural meaning, and audio; learner records themselves saying it

3.3 Adaptive Learning Engine

The adaptive engine is the bridge between Component 1 (Learning App) and Component 3 (AI Module):

- Performance Tracker: logs every interaction (correct/incorrect, response time, pronunciation score, skip rate) per vocabulary unit and lesson type
- Difficulty Adjustment: Bayesian knowledge tracing model estimates per-word mastery probability; words below 80% mastery are re-queued with increased frequency
- Path Optimizer: selects next lesson based on weighted combination of: weakest skill area, spaced repetition schedule, cultural content freshness, and user engagement pattern
- Tonal Priority System: Medumba's five tones (high, mid, low, rising, falling) are tracked independently; the system identifies which tonal patterns the learner struggles with and generates targeted tonal drills

3.4 Offline Architecture (Mobile)

Critical for learners in Cameroon with intermittent connectivity:

- On first launch, the Flutter app downloads the current lesson pack (next 10 lessons), associated audio files, and full dictionary cache to local SQLite/Hive storage
- Pronunciation recordings are cached locally and uploaded for AI scoring when connectivity returns; approximate local scoring provides immediate feedback
- Progress data syncs bidirectionally with conflict resolution (last-write-wins with server authority on scoring)
- Dictionary is fully available offline including all audio pronunciation files (approximately 150MB compressed for full corpus)

3.5 Web vs. Mobile Feature Parity

Feature	Web (React.js PWA)	Mobile (Flutter)
Lessons & gamification	Full	Full
Pronunciation recording	Via Web Audio API	Native mic access
Offline access	Limited (PWA cache)	Full offline mode
Push notifications	Browser notifications	Native push (FCM/APNS)
Audio playback	Streaming from CDN	Local cache + streaming
Dictionary	Full (online)	Full (offline-capable)
Payment/subscription	Stripe web checkout	In-app purchase + Stripe

4. COMPONENT 2: DICTIONARY MODULE & COMMUNITY WORD HARVESTING

The Dictionary Module is both the platform's **permanent preservation artifact** and its **living linguistic engine**. Unlike a static dictionary, it is designed to grow continuously through structured community contributions while maintaining rigorous linguistic validation. The MVP dictionary contains 4,000+ Medumba–French–English entries with 600+ authenticated elder pronunciation files — the largest structured digital Medumba lexicon ever created.

4.1 Dictionary Entry Data Model

Each dictionary entry is a rich document containing:

Field	Type	Description
entry_id	UUID	Unique identifier
medumba_word	String	Medumba headword in standard orthography
tonal_notation	String	IPA tonal marking (5-tone: H, M, L, R, F)
french_translation	String	French equivalent(s)
english_translation	String	English equivalent(s)
part_of_speech	Enum	Noun (with class prefix), verb, adjective, adverb, pronoun, conjunction, interjection
noun_class	Integer (1–19)	Bantu noun class assignment (for nouns)
pronunciation_files[]	Array<AudioRef>	References to elder pronunciation recordings in object storage; multiple elders per word
example_sentences[]	Array<Object>	Medumba sentence + French/English translation + audio
cultural_context	Text	Usage notes, cultural significance, proverbs containing this word
cultural_pillar	Enum	Heritage | Identity | Unity | Resilience | Hope & Future | Intergenerational Connection
etymology	Text (optional)	Word origin, cognates in related Grassfields languages
related_words[]	Array<UUID>	Cross-references to synonyms, antonyms, derived forms
source	Enum	Elder recording session | Linguistic corpus | Community submission | Metchezin validation
validation_status	Enum	Draft | Submitted | Under Review | Validated | Published
validated_by	String	Name of linguistic validator (typically Cultural Director)
created_at / updated_at	Timestamp	Audit trail
tone_pattern	String (regex pattern)	Machine-readable tonal pattern e.g., "H-L" for high-low; enables algorithmic generation of tonal drills and minimal pair identification

orthographic_variants	Array<String>	Alternate spellings with/without tone marks (e.g., ["bá", "ba-high", "ba1"]); critical for search and community submission matching
minimal_pairs	Array<UUID>	References to dictionary entries differing only in tone (e.g., *bá* vs. *bà*); powers tonal discrimination exercises

4.2 Community Word Harvesting System

The Word Harvesting feature is the dictionary's growth engine. It enables any community member — elders in Cameroon, diaspora families in New York, youth at ROOTS & WINGS sessions, festival attendees at NKÒNÎ — to **submit words, phrases, or proverbs they know but that are not yet in the dictionary**. This is critical because many Medumba words exist only in oral tradition and have never been documented.

Submission Flow:

- Contributor opens "Add a Word" from the dictionary or Learning App
- Enters the Medumba word (text), provides French and/or English meaning, optionally records audio pronunciation from their own voice
- Optionally adds context: "I heard my grandmother use this word when..."; tags a cultural pillar; marks as proverb/idiom/specialized vocabulary
- Submission enters the Moderation Queue with status "Submitted"

Validation Workflow:

- Submitted entries are visible to the Cultural Director and designated linguistic validators
- Validators review for: (a) authenticity — is this a real Medumba word?; (b) accuracy — is the meaning correct?; (c) tonal notation — can correct tonal marking be assigned?; (d) duplication — does this entry already exist under a variant spelling?
- If validated, the Cultural Director records or assigns an official elder pronunciation file
- Entry moves to "Published" and becomes visible in the public dictionary and available to the Learning App lesson generator
- Contributor receives credit and a "Language Guardian" badge in the gamification system

Harvesting at Scale: The NKÒNÎ Booth Model:

- The "Stump the Dictionary" activity at the NKÒNÎ Festival is a structured word harvesting event: elders deliberately try to submit obscure words the dictionary doesn't have
- Each submission is logged in real-time to the "NKÒNÎ 2026 Contributions" queue for post-festival validation by the Cultural Director in Cameroon
- This creates a recurring annual word harvesting cycle tied to the community's flagship cultural event

4.3 Dictionary Search & Browse

- Full-text search across Medumba, French, and English fields with fuzzy matching for spelling variations
- Browse by: noun class, cultural pillar, part of speech, recently added, most played audio
- Audio playback: tap any entry to hear elder pronunciation; long-press for slow playback; compare multiple elder recordings of the same word
- Cross-reference navigation: tap related words to explore linguistic connections
- Bookmarking: users save words to personal study lists that feed into the Learning App's spaced repetition queue

4.4 Linguistic Corpus Export

For academic and research use, the dictionary supports structured export:

- XML export conforming to TEI (Text Encoding Initiative) standards for linguistic corpora
- JSON-LD export for linked data / semantic web applications
- CSV export for spreadsheet-based analysis
- Audio corpus export with aligned transcriptions for NLP training data (restricted to authorized researchers under IP Framework)

5. COMPONENT 3: AI MODULE, CHATBOT & TEACHANYTHING.AI INTEGRATION

The AI Module is the intelligence layer that powers the platform's most differentiated capabilities. It is structured as a set of **microservices** behind a unified AI API, consumed by both the Learning App and the external TeachAnything.ai integration.

5.1 AI Service Architecture

Service	Technology	Function
Pronunciation Scoring Engine	Python, Whisper (fine-tuned), PyTorch, librosa	Accepts learner audio → extracts pitch contours (F0) → compares against elder reference recordings → returns tonal accuracy score (0–100) + specific tone-error feedback per syllable
NLP Engine	Python, spaCy (custom pipeline), HuggingFace Transformers	Medumba tokenization, morphological analysis (noun class parsing), part-of-speech tagging, sentence segmentation; trained on validated dictionary corpus
Adaptive Learning Service	Python, scikit-learn	Bayesian knowledge tracing, spaced repetition scheduler (modified SM-2), lesson path optimization, tonal difficulty modeling
Content Generation Pipeline	Python, Claude API (Anthropic)	Generates example sentences, cultural context notes, lesson scaffolding; all output reviewed by Cultural Director before publication
Chatbot Service	Python, LangChain, Claude API	Conversational Medumba tutor: answers language questions, provides vocabulary practice, explains grammar and tonal rules, code-switches between Medumba/French/English
TeachAnything.ai Connector	Python, REST API	Bridges Medumba.ai content engine to the TeachAnything.ai open-access platform; serves lesson modules and chatbot capabilities through TeachAnything's interface

5.2 Pronunciation Scoring Engine — Deep Dive

This is the platform's most technically challenging component. Medumba has five lexical tones (high, mid, low, rising, falling) where a single pitch change transforms meaning entirely. Standard speech recognition models (Whisper, Wav2Vec) are trained to be **pitch-invariant**—they explicitly discard fundamental frequency information to generalize across speakers. For Medumba, where pitch *is* the phoneme, this creates a fundamental architectural mismatch.

5.2.1 Core Technical Challenge: The Pitch-Invariance Problem

Automatic Speech Recognition (ASR) systems optimize for **phoneme recognition** independent of speaker pitch. A Mandarin ASR model recognizes "mā" whether spoken by a bass-voiced male (F0

~80Hz) or a child (F0 ~300Hz). This is achieved through **delta features** (pitch derivatives) and **speaker normalization** that strip absolute F0.

For Medumba pronunciation scoring, we need the **opposite**: a model hyper-sensitive to F0 contours while invariant to speaker identity. This requires architectural choices distinct from standard ASR fine-tuning.

5.2.2 Recommended Architecture: Prosody-Aware Neural Scoring

Rather than fine-tuning Whisper (pitch-invariant), we implement a **two-stage prosody-aware pipeline**:

Stage	Component	Technology	Function
1	Phonetic Forced Aligner	Montreal Forced Aligner (MFA) with custom Medumba acoustic model	Segments learner audio into syllable boundaries; required because DTW requires pre-segmentation
2	Tone Classifier	CNN-LSTM with F0 concatenation	Per-syllable 5-class classification (H/M/L/R/F) on pitch-normalized contours

Pipeline Detail:

- **Audio Ingestion**: 16kHz mono WAV normalization; VAD removal of silence
- **Syllable Segmentation**: MFA with Medumba-specific acoustic model (trained on elder corpus). Critical: Without this, DTW alignment is impossible.
- **F0 Extraction**: CREPE algorithm (deep learning-based pitch tracking) extracts fundamental frequency at 10ms intervals
- **Speaker Normalization**: Convert absolute F0 to relative semitone deviation from speaker's mean pitch. Essential because elder voices (low F0) and youth learners (high F0) occupy different absolute ranges but identical relative tone categories.
- **Contour Quantization**: Map continuous F0 to discrete tone categories using tone space clustering validated by Georgetown phonological analysis
- **Error Detection**: Compare learner's normalized contour against elder reference; identify syllable-level tone mismatches
- **Feedback Generation**: Natural language feedback with specific pitch direction cues (e.g., "Second syllable: your pitch rose 2 semitones, but Medumba rising tone requires 4-5 semitones—try a steeper rise")

5.2.3 Training Data Strategy: Adapted for our low-resource corpus

Assuming approximately 600 elder recordings in our current corpus (~1,200-1,800 syllables estimated). For robust 5-class tone classification with cross-speaker generalization:

Requirement	Current	Gap	Mitigation
Labeled syllables	~1,500	Need 5,000+	Cross-lingual transfer from Fe'fe', Ghomálá'
Speaker diversity	~10 elders	Need 25+ speakers	Community recording campaign (NKÒNÎ 2026)
Negative examples	Pilot testing only	Need 1,000+ mispronunciations	Synthetic tone injection (see below)
Phonetic coverage	Common vocab only	Need tone in all contexts	Targeted harvesting of minimal pairs

Data Augmentation Techniques:

- **Pitch Shifting**: ±3 semitones on elder recordings to simulate learner variability without altering tone relationships
- **Synthetic Tone Injection**: Apply Medumba tone patterns to French/English speech via PSOLA (Pitch-Synchronous Overlap and Add) to generate artificial training examples with known tone labels
- **Time Stretching**: ±20% speed variation to improve robustness to learner speaking rate

5.2.4 Fallback Strategy: Human-in-the-Loop Evaluation

If by Month 6 the corpus contains <2,000 labeled syllables with <15 speakers, automated scoring will lack reliability. Contingency: Implement "Cultural Director Review Queue" where learner recordings are batched for human tonal evaluation, with AI providing candidate labels (not final scores). This preserves learning loop functionality while building training data for future automation.

5.2.5 Integration with Adaptive Learning

Per-syllable tone accuracy feeds into the Tonal Mastery Model—a separate Bayesian knowledge tracer tracking H/M/L/R/F individually. The system identifies which tonal contrasts a learner struggles with (e.g., High vs. Rising confusion) and generates targeted minimal-pair drills from dictionary entries.

5.3 Chatbot Architecture

The Medumba.ai chatbot serves as an AI-powered conversational Medumba tutor, available both within the native platform and through TeachAnything.ai:

Architecture:

- LLM Backbone: Claude API (Anthropic) for natural language understanding and generation; selected for multilingual capability across Medumba/French/English

- RAG (Retrieval-Augmented Generation): chatbot queries are first matched against the dictionary database and validated lesson content via vector similarity search; retrieved context is injected into the LLM prompt to ensure factual accuracy
- Conversation Memory: per-session conversation history maintained for coherent multi-turn dialogue; long-term user context (learning level, weak areas) injected from the adaptive learning service
- Guardrails: all chatbot outputs about Medumba vocabulary, grammar, or cultural content are constrained to validated dictionary entries; the chatbot explicitly states when it is uncertain and recommends consulting the Cultural Director
- Code-Switching: the chatbot naturally switches between Medumba, French, and English based on the user's language preference and proficiency level

Chatbot Use Cases:

- "How do I say 'thank you' in Medumba?" → returns dictionary entry with audio, tonal notation, and cultural context
- "Explain the difference between tone 1 and tone 3" → generates explanation with audio examples from the dictionary
- "Quiz me on food vocabulary" → generates interactive vocabulary drill from the learner's current lesson path
- "What does this proverb mean?" → retrieves proverb entry, provides literal translation and cultural interpretation
- "Help me practice greetings" → role-play dialogue with pronunciation feedback

Tonal Hallucination Guardrails:

Standard LLMs (Claude, GPT-4) are not tone-aware for low-resource languages. Without explicit constraints, the chatbot will generate Medumba words with **plausible but incorrect tone patterns**—a catastrophic error for a language learning tool.

Mitigation Strategy:

- **Dictionary-Grounded Generation**: All Medumba words in chatbot responses must be retrieved from the validated dictionary (RAG), never generated token-by-token by the LLM
- **Tone Validation Layer**: Post-process all LLM outputs containing Medumba text through the NLP engine to verify tones exist in dictionary; flag mismatches for human review
- **Phonetic Prompting**: Include IPA tone notation in RAG context sent to LLM (e.g., "The word for 'water' is **ndá** [nda˥], high tone") to increase tone awareness in context

5.4 TeachAnything.ai Integration

TeachAnything.ai (www.teachanything.ai) serves as the **open-access front door** of the Medumba.ai initiative — the permanently free interface through which learners worldwide first encounter the platform. Medumba.ai provides the content engine; TeachAnything.ai provides the delivery layer.

Integration Architecture:

- Medumba.ai exposes a dedicated Integration API (REST) that serves: lesson module content (JSON), dictionary lookups, chatbot conversational endpoints, and pronunciation audio streams
- TeachAnything.ai consumes these endpoints to render the bilingual Medumba chatbot experience within its own UI framework
- Authentication: API key-based access; TeachAnything.ai authenticated as a trusted integration partner under the IP Framework

- Data Sovereignty: all Medumba.ai data remains on Medumba.ai infrastructure; TeachAnything.ai receives read-only API responses and does not cache, store, or use Medumba content for any purpose other than real-time display to end users
- Content Pipeline: GWU student research associates pilot and document Medumba lesson modules on TeachAnything.ai, validating the integration in a live educational setting
- Independence Guarantee: the Medumba.ai platform maintains a fully independent delivery pathway (native web + mobile apps) capable of serving all learners without reliance on TeachAnything.ai

5.5 Low-Resource Tonal Language Training

Medumba presents a **compound low-resource problem**: (1) limited text corpus, (2) limited audio corpus, (3) tonal complexity requiring precise audio-text alignment, and (4) no pre-trained models in the language family.

Why Standard NLP Recipes Fail:

- **Text-only pretraining** (BERT-style): Loses the tonal information that distinguishes 40%+ of Medumba lexical items
- **Speech-only pretraining** (wav2vec-style): Learns acoustic features but no semantic grounding
- **Standard multitask learning**: Tonal languages require **tri-modal** alignment (audio pitch + phonetic text + semantic meaning)

Recommended Training Paradigm:

1. **Phase 1**: Cross-lingual transfer from Fe'fe' and Ghomálá' (related Grassfields languages with more documentation)
2. **Phase 2**: Joint audio-text contrastive learning on Medumba elder recordings (learn tone-to-meaning mappings)
3. **Phase 3**: Instruction tuning on dictionary-derived QA pairs (ground language-specific knowledge)

Compute: This requires 10-20x more GPU hours than standard fine-tuning. Budget 500+ A100-hours for the pronunciation scoring engine alone.

6. DATA ARCHITECTURE & STORAGE

6.1 Database Design

Store	Technology	Contents
Primary Relational DB	PostgreSQL 16	Users, authentication, subscription/billing, lesson progress, XP/gamification state, learning analytics, moderation queue
Document Store	MongoDB 7	Dictionary entries (rich document model), lesson packs (JSON), chatbot conversation logs, community word submissions
Audio/Media Store	S3-compatible (MinIO or AWS S3)	Elder pronunciation files (WAV/MP3), learner pronunciation recordings, lesson narration audio, cultural story illustrations
Cache Layer	Redis 7	Session tokens, hot dictionary lookups, leaderboard state, rate limiting counters, real-time pronunciation scoring queue
Vector Store	pgvector (PostgreSQL extension)	Embedding vectors for dictionary entries and lesson content; powers RAG retrieval for chatbot and semantic search
Search Index	Meilisearch	Full-text trilingual search index over dictionary entries with fuzzy matching, typo tolerance, and faceted filtering

6.2 Audio Storage Architecture

Audio is the platform's most critical and largest data asset:

- **Storage tiers:** Elder recordings stored as lossless WAV (archival master) + compressed MP3 (serving copy) + spectrogram image (visual reference for tonal analysis)
- **Naming convention:** Audio files organized by entry_id with consistent naming: {entry_id}/{elder_name}_{recording_date}.{format}
- **Triple redundancy:** All audio replicated to three locations: primary cloud, offline encrypted storage in Cameroon, U.S. Embassy Yaoundé shared repository
- **CDN delivery:** CDN-served MP3 copies cached at edge locations for low-latency playback; aggressive caching for the 200 most-accessed entries

6.3 Backup & Disaster Recovery

- PostgreSQL: continuous WAL archiving + daily full backups; 30-day retention; cross-region replication
- MongoDB: replica set with 3 nodes; daily mongodump to encrypted offsite storage
- Audio corpus: immutable object storage with versioning; quarterly integrity verification against checksum manifest
- Recovery Time Objective (RTO): 4 hours; Recovery Point Objective (RPO): 1 hour

7. API DESIGN

All three components communicate through a unified RESTful API. The API is versioned (v1), authenticated via JWT, and rate-limited per user tier.

7.1 Learning App API

Endpoint	Method	Description
`/api/v1/lessons/next`	GET	Returns next lesson pack based on adaptive path
`/api/v1/lessons/{id}/complete`	POST	Submits lesson completion with per-exercise scores
`/api/v1/pronunciation/score`	POST	Uploads audio recording; returns tonal accuracy score + feedback
`/api/v1/progress`	GET	Returns user progress: XP, streak, level, per-word mastery
`/api/v1/leaderboard`	GET	Returns weekly leaderboard (global + friends)
`/api/v1/review/queue`	GET	Returns spaced repetition review queue for current session

7.2 Dictionary API

Endpoint	Method	Description
`/api/v1/dictionary/search`	GET	Trilingual full-text search with fuzzy matching; params: q, lang, noun_class, pillar
`/api/v1/dictionary/{id}`	GET	Returns complete dictionary entry with audio references
`/api/v1/dictionary/{id}/audio`	GET	Streams pronunciation audio file (elder recording)
`/api/v1/dictionary/submit`	POST	Community word submission; creates entry in moderation queue
`/api/v1/dictionary/harvest/queue`	GET	Returns pending word submissions for validators (auth: validator role)
`/api/v1/dictionary/harvest/{id}/validate`	PUT	Validates or rejects a community submission (auth: validator role)
`/api/v1/dictionary/export`	GET	Exports dictionary corpus in XML (TEI), JSON-LD, or CSV format (auth: researcher role)

7.3 AI Module API

Endpoint	Method	Description
`/api/v1/ai/chat`	POST	Chatbot endpoint; accepts message + conversation history; returns response
`/api/v1/ai/pronunciation/analyze`	POST	Deep pronunciation analysis; returns per-syllable tonal breakdown
`/api/v1/ai/adaptive/path`	GET	Returns personalized learning path based on current mastery model
`/api/v1/ai/generate/sentence`	POST	Generates example sentence for a given word (output flagged for review)
`/api/v1/ai/tonal/drill`	GET	Returns targeted tonal drill based on user's weak tone patterns

7.4 TeachAnything.ai Integration API

Endpoint	Method	Description
`/api/v1/external/teach/lessons`	GET	Returns lesson modules formatted for TeachAnything.ai consumption
`/api/v1/external/teach/chat`	POST	Proxied chatbot endpoint for TeachAnything.ai users

`/api/v1/external/teach/dictionary`	GET	Read-only dictionary search for TeachAnything.ai interface
`/api/v1/external/teach/audio/{id}`	GET	Streams pronunciation audio to TeachAnything.ai (CDN-cached)

8. INFRASTRUCTURE & DEPLOYMENT

8.1 Cloud Architecture

- Primary hosting: managed Kubernetes cluster (GKE, EKS, or equivalent) with auto-scaling node pools
- AI workloads: dedicated GPU-enabled nodes for pronunciation scoring and NLP inference; spot/preemptible instances for batch processing (content generation, corpus analysis)
- CDN: global content delivery for audio files, static assets, and web app bundle; Cloudflare or AWS CloudFront
- DNS & load balancing: managed load balancer with SSL termination; geographic routing for optimal latency
- Monthly infrastructure cost: approximately $389/month at MVP scale, with scalability buffer for launch traffic spikes

8.2 CI/CD Pipeline

- Source control: Git (GitHub) with branch protection; feature branches → pull request → code review → merge to main
- Automated testing: unit tests (Jest for JS, pytest for Python), integration tests against staging database, end-to-end tests (Cypress for web)
- Build: Docker image build on merge to main; tagged with commit SHA and semantic version
- Deploy: automated rollout to staging environment; manual promotion to production after QA; blue-green deployment for zero-downtime releases
- Monitoring: application performance monitoring (APM), error tracking, uptime checks, AI model performance dashboards

8.3 Environment Strategy

Environment	Purpose	Access
Development	Local developer machines + shared dev cluster	Dev team only
Staging	Pre-production mirror; integration testing, QA	Dev team + internal testers
Production	Live platform serving end users	Public (authenticated)
AI Sandbox	Isolated environment for AI model training and evaluation	AI team + research partners

9. SECURITY ARCHITECTURE

- JWT-based authentication with refresh token rotation; bcrypt password hashing; optional social OAuth (Google, Apple)
- Role-based access control (RBAC): Learner, Contributor, Validator, Teacher, Admin, Researcher, External Partner (TeachAnything.ai)
- All API communication over TLS 1.3; HSTS enforced; certificate pinning on mobile app
- Input validation and sanitization on all API endpoints; parameterized database queries to prevent injection
- Audio uploads scanned for malicious content; file type and size validation enforced
- PII handling: minimal personal data collection; email + display name only for basic accounts; payment data handled by Stripe (PCI DSS compliant, never touches Medumba.ai servers)
- Data encryption: at-rest encryption for all databases and object storage; AES-256
- IP Framework compliance: TeachAnything.ai integration enforces read-only access; no data caching, no training data extraction; API key rotation every 90 days
- Annual security audits and penetration testing; vulnerability scanning integrated into CI/CD pipeline
- Incident response: documented runbook; automated alerting; 4-hour response SLA for critical security events

10. INTEGRATION MAP: HOW THE THREE COMPONENTS CONNECT

The three components are not isolated products — they form an integrated ecosystem where data flows continuously between them:

Dictionary → Learning App: Dictionary entries are the raw material from which lessons are generated. When a new word is validated and published in the Dictionary Module, it becomes available to the Learning App's lesson generation pipeline. The adaptive learning engine draws vocabulary units, pronunciation files, example sentences, and cultural context directly from the dictionary database.

Learning App → Dictionary: Learner interaction data flows back to enrich the dictionary. When learners consistently struggle with a word's pronunciation, the system flags it for additional elder recordings or clearer tonal notation. The "Bookmark" feature creates usage analytics showing which words are most demanded by learners, guiding the Cultural Director's prioritization of word harvesting.

Dictionary → AI Module: The validated dictionary corpus serves as the primary training data for the NLP engine and the RAG knowledge base for the chatbot. Every validated entry — with its tonal notation, translations, cultural context, and audio — is embedded into the vector store to power chatbot retrieval and semantic search.

AI Module → Learning App: The AI Module provides four critical services to the Learning App: (1) pronunciation scoring for every learner recording; (2) adaptive path optimization determining which lesson comes next; (3) tonal drill generation targeting the learner's specific weak spots; (4) example sentence generation enriching lesson content.

AI Module → Dictionary: The content generation pipeline can propose new example sentences, cultural context notes, and cross-references for existing dictionary entries. All AI-generated content enters the moderation queue for Cultural Director validation before publication — AI augments human expertise but never publishes autonomously.

AI Module → TeachAnything.ai: The chatbot service and lesson content are exposed through the Integration API to TeachAnything.ai. This creates the "open-access front door": a permanently free bilingual Medumba chatbot accessible to anyone through an established American educational technology platform, with no login or subscription required.

Community Word Harvesting → All Components: Word submissions from community members, festival attendees, and youth programs flow into the Dictionary Module's moderation queue. Once validated, they propagate to the Learning App (new vocabulary available for lessons), the AI Module (expanded training data and chatbot knowledge), and the TeachAnything.ai integration (richer content available through the open-access layer).

11. DEVELOPMENT ROADMAP

Phase / Timeline	Deliverables	Components Active
Phase 1: Foundation Months 1–3	Dictionary database schema + import pipeline; 4,000+ entries loaded; audio storage architecture; basic dictionary web UI (search/browse/play); API Gateway + authentication system; development/staging environments	Dictionary Module (core)
Phase 2: Dictionary MVP Months 2–6	Complete dictionary UI (web + mobile); community word submission form + moderation queue; pronunciation file CDN delivery; trilingual search with fuzzy matching; XML/JSON export; dictionary fully offline-capable on mobile	Dictionary Module (full)
Phase 3: AI Core Months 3–9	Pronunciation scoring engine v0.1 (rule-based F0 comparison; limited to 200 most common words); NLP engine (tokenization, POS tagging); tonal text normalization pipeline; cross-lingual transfer from related Bantu languages ; Pronunciation scoring v1.0 (neural tone classifier); adaptive learning integration; chatbot v0.5 (dictionary-grounded only); Chatbot v1.0 with tone-aware generation constraints	AI Module (core)
Phase 4: Learning App Months 6–12	Lesson module system + 100 structured lessons + 1,200 vocabulary units; gamification engine (XP, streaks, badges, leaderboards); pronunciation practice with live AI scoring; spaced repetition review system; cultural storytelling modules; web app (React) + mobile app (Flutter) launch	Learning App (full)
Phase 5: Integration Months 9–15	TeachAnything.ai connector + bilingual chatbot deployment; end-to-end integration testing across all three components; teacher marketplace beta (15-teacher network); subscription billing (Stripe); word harvesting at NKÒNÎ 2026	All three + TeachAnything.ai
Phase 6: Launch & Scale Months 12–18	Public launch (Yaoundé, Bangangté, NKÒNÎ Festival); performance optimization + auto-scaling validation; security audit + penetration testing; analytics dashboards + impact reporting; post-launch iteration based on user feedback	Full platform

End of Document